

```
from abc import abstractmethod
```

```
"""
```

在讲解背包问题的时候，我们都是按照如下五部来逐步分析，相信大家也体会到，把这五部都搞透了，算是对动规来理解深入了。

确定dp数组（dp table）以及下标的含义

确定递推公式

dp数组如何初始化

确定遍历顺序

举例推导dp数组

```
"""
```

```
class DynamicAlgo:
```

```
    @abstractmethod
```

```
    def climb_stairs(n: int) -> int:
```

```
        """
```

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

示例 2:

输入: $n = 3$

输出: 3

```
        """
```

```
        if n <= 2:
```

```
            return n
```

```
        nums = [1, 2]
```

```
        for i in range(2, n):
```

```
            tmp = nums[1]
```

```
            nums[1] = nums[1] + nums[0]
```

```
            nums[0] = tmp
```

```
        return nums[1]
```

```
    @abstractmethod
```

```
    def climbing_stairs(self, n: int, m: int) -> int:
```

```
        """
```

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬至多 m ($1 \leq m < n$) 个台阶。你有多少种不同的方法可以爬到楼顶呢？

```
        """
```

```
        dp = [0] * (n + 1)
```

```
        dp[0] = 1
```

```
        for i in range(1, n + 1):
```

```
            for j in range(1, m + 1):
```

```
                if i >= j:
```

```
                    dp[j] += dp[j - i]
```

```
        return dp[n]
```

```
    @abstractmethod
```

```
    def minCostClimbingStairs(self, cost: list[int]) -> int:
```

```
        """
```

给你一个整数数组 `cost`，其中 `cost[i]` 是从楼梯第 i 个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个或者两个台阶。

你可以选择从下标为 0 或下标为 1 的台阶开始爬楼梯。请你计算并返回达到楼梯顶部的最低花费。

示例 1:

输入: `cost = [10,15,20]`

输出: 15

示例 2:

输入: `cost = [1,100,1,1,1,100,1,1,100,1]`

输出: 6

"""

```
dp = [0] * (len(cost) + 1)
```

```
dp[0] = 0 # 初始值, 表示从起点开始不需要花费体力
```

```
dp[1] = 0 # 初始值, 表示经过第一步不需要花费体力
```

```
for i in range(2, len(cost) + 1):
```

```
    # 在第i步, 可以选择从前一步 (i-1) 花费体力到达当前步, 或者从前两步 (i-2) 花费体力到达当前步
```

```
    # 选择其中花费体力较小的路径, 加上当前步的花费, 更新dp数组
```

```
    dp[i] = min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2])
```

```
return dp[len(cost)] # 返回到达楼顶的最小花费
```

```
@abstractmethod
```

```
def unique_paths(self, m: int, n: int) -> int:
```

```
    """
```

一个机器人位于一个 $m \times n$ 网格的左上角。机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角。问总共有多少条不同的路径?

示例 1:

输入: `m = 3, n = 7`

输出: 28

示例 2:

输入: `m = 3, n = 2`

"""

```
f = [1] * n
```

```
for i in range(1, m):
```

```
    for j in range(1, n):
```

```
        f[j] += f[j-1]
```

```
return f[n - 1]
```

```
@abstractmethod
```

```
def uniquePathswithObstacles(self, obstacleGrid):
```

```
    """
```

现在考虑网格中有障碍物。那么从左上角到右下角将会有多少条不同的路径?

```
    """
```

```
if obstacleGrid[0][0] == 1:
```

```
    return 0
```

```
m, n = len(obstacleGrid), len(obstacleGrid[0])
```

```
dp = [0] * n # 创建一个一维列表用于存储路径数
```

```
# 初始化第一行的路径数
```

```
for j in range(n):
```

```
    if obstacleGrid[0][j] == 1:
```

```
        break
```

```

dp[j] = 1

# 计算其他行的路径数
for i in range(1, m):
    if obstacleGrid[i][0] == 1:
        dp[0] = 0
    for j in range(1, n):
        if obstacleGrid[i][j] == 1:
            dp[j] = 0
            continue

        dp[j] += dp[j - 1]

return dp[-1] # 返回最后一个元素，即终点的路径数

```

@abstractmethod

```

def integerBreak(self, n: int) -> int:
    """

```

给定一个正整数 n ，将其拆分为至少两个正整数的和，并使这些整数的乘积最大化。返回你可以获得的最大乘积。

示例 1:

输入: 2

输出: 1

示例 2:

输入: 10

输出: 36

"""

```

if n <= 3:
    return 1 * (n - 1) # 对于n小于等于3的情况，返回1 * (n - 1)

```

```

dp = [0] * (n + 1) # 创建一个大小为n+1的数组来存储最大乘积结果
dp[1] = 1 # 当n等于1时，最大乘积为1
dp[2] = 2 # 当n等于2时，最大乘积为2
dp[3] = 3 # 当n等于3时，最大乘积为3

```

从4开始计算，直到n

```

for i in range(4, n + 1):
    # 遍历所有可能的切割点
    for j in range(1, i // 2 + 1):
        # 计算切割点j和剩余部分(i - j)的乘积，并与之前的结果进行比较取较大值
        dp[i] = max(dp[i], dp[i - j] * dp[j])

```

```

return dp[n] # 返回整数拆分的最大乘积结果

```

@abstractmethod

```

def num_trees(self, n: int) -> int:
    """

```

给你一个整数 n ，求恰由 n 个节点组成且节点值从 1 到 n 互不相同的二叉搜索树有多少种？返回满足题意的二叉搜索树的种数。

示例 1:

输入: $n = 3$

输出: 5

"""

```

dp = [0] * (n + 1)

```

```

dp[0], dp[1] = 1, 1

for i in range(2, n + 1):
    for j in range(1, i + 1):
        dp[i] += dp[i - 1] * dp[j - i]
    return dp[n]

@abstractmethod
def bag_weight_problem(self, weight: list[int], value: list[int], bagweight:
int)-> int:

    dp = [[0] *(bagweight + 1) for _ in range(len(weight))]
    # 即dp[i][j] 表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。

    for j in range(weight[0], bagweight + 1):
        dp[0][j] = value[0]

    for i in range(1, len(weight)):
        for j in range(bagweight + 1):
            if j < weight[i]:
                dp[i][j] = dp[i-1][j]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] +
value[i])
    return dp[len(weight) - 1][bagweight]

@abstractmethod
def bag_weight_problem2(self, weight: list[int], value: list[int], bagweight:
int)-> int:
    dp = [0] *(bagweight + 1)
    for i in range(len(weight)): # 遍历物品
        for j in range(bagweight, weight[i] - 1, -1): # 遍历背包容量
            # 01背包内嵌的循环是从大到小遍历，为了保证每个物品仅被添加一次。
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i])
    return dp[bagweight]

@abstractmethod
def can_partition(self, nums: list[int]) -> bool:
    """
    给定一个只包含正整数的非空数组。是否可以将这个数组分割成两个子集，使得两个子集的元素和相
    等。

    注意：每个数组中的元素不会超过 100 数组的大小不会超过 200

    示例 1:
    输入: [1, 5, 11, 5]
    输出: true
    """
    if sum(nums) % 2 != 0:
        return False
    target = sum(nums) // 2
    dp = [0] * (target + 1)
    for num in nums:
        for j in range(target, num-1, -1):
            dp[j] = max(dp[j], dp[j-num] + num)
    return dp[-1] == target

```

```
@abstractmethod
```

```
def last_stone_weight2(self, stones: list[int]) -> int:
```

```
    """
```

有一堆石头，每块石头的重量都是正整数。

每一回合，从中选出任意两块石头，然后将它们一起粉碎。假设石头的重量分别为 x 和 y ，且 $x \leq y$ 。那么粉碎的可能结果如下：

如果 $x == y$ ，那么两块石头都会被完全粉碎；

如果 $x \neq y$ ，那么重量为 x 的石头将会完全粉碎，而重量为 y 的石头新重量为 $y-x$ 。

最后，最多只会剩下一块石头。返回此石头最小的可能重量。如果没有石头剩下，就返回 0。

示例：

输入：[2,7,4,1,8,1]

输出：1

```
    """
```

```
    total_sum = sum(stones)
```

```
    target = total_sum // 2
```

```
    dp = [0] * (target + 1)
```

```
    for stone in stones:
```

```
        for j in range(target, stone - 1, -1):
```

```
            dp[j] = max(dp[j], dp[j - stone] + stone)
```

```
    return total_sum - 2 * dp[-1]
```

```
@abstractmethod
```

```
def find_target_get_sum_ways(self, nums: list[int], target: int) -> int:
```

```
    """
```

给你一个非负整数数组 `nums` 和一个整数 `target`。向数组中的每个整数前添加 '+' 或 '-'，然后串联起所有整数，可以构造一个表达式：

例如，`nums = [2, 1]`，可以在 2 之前添加 '+'，在 1 之前添加 '-'，然后串联起来得到表达式 "+2-1"。

返回可以通过上述方法构造的、运算结果等于 `target` 的不同 表达式 的数目。

示例 1：

输入：`nums = [1,1,1,1,1]`，`target = 3`

输出：5

假设加法的总和为 x ，那么减法对应的总和就是 $sum - x$ 。所以我们要求的是 $x - (sum - x) = target$ 。 $x = (target + sum) / 2$

```
    """
```

```
    total_sum = sum(nums)
```

```
    if abs(target) > total_sum:
```

```
        return 0
```

```
    if (target + total_sum) % 2 != 0:
```

```
        return 0
```

```
    bagsize = (target + total_sum) // 2
```

```
    dp = [0] * (bagsize + 1)
```

`dp[i][j]`：使用 下标为`[0, i]`的`nums[i]`能够凑满`j`（包括`j`）这么大量容量的包，有`dp[i][j]`种方法。

```
    dp[0] = 1
```

```
    for i in range(len(nums)):
```

```
        for j in range(bagsize, nums[i] - 1, -1):
```

```
            dp[j] += dp[j - nums[i]]
```

```
    return dp[bagsize]
```

```
@abstractmethod
```

```
def complete_pack_problem(self, weight: list[int], value: list[int],  
bagweight: int) -> int:
```

```
"""
有N件物品和一个最多能背重量为w的背包。第i件物品的重量是weight[i]，得到的价值是
value[i]。
每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。
"""
```

```
dp = [0] * (bagweight + 1)
for i in range(len(weight)): # 遍历物品
    for j in range(weight[i], bagweight + 1): # 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i])
return dp[bagweight]
```

@abstractmethod

```
def money_change(self, amount: int, coins: list[int]) -> int:
    """
```

给你一个整数数组 `coins` 表示不同面额的硬币，另给一个整数 `amount` 表示总金额。
请你计算并返回可以凑成总金额的硬币组合数。如果任何硬币组合都无法凑出总金额，返回 `0`。
假设每一种面额的硬币有无限个。

示例 1:

输入: `amount = 5, coins = [1, 2, 5]`

输出: 4

```
"""
```

```
dp = [0] * (amount + 1)
dp[0] = 1
for i in range(len(coins)): # 组合数，不考虑顺序
    for j in range(coins[i], amount + 1):
        dp[j] += dp[j - coins[i]]
return dp[amount]
```

@abstractmethod

```
def combination_sum4(self, nums: list[int], target: int) -> int:
    """
```

给你一个由 不同 整数组成的数组 `nums`， 和一个目标整数 `target`。请你从 `nums` 中找出并返回总和为 `target` 的元素组合的个数。

题目数据保证答案符合 32 位整数范围。

示例 1:

输入: `nums = [1,2,3], target = 4`

输出: 7

```
"""
```

```
dp = [0] * (target + 1)
dp[0] = 1
for i in range(1, target + 1): # 遍历背包，排列数，考虑顺序
    for j in range(len(nums)): # 遍历物品
        if i - nums[j] >= 0:
            dp[i] += dp[i - nums[j]]
return dp[target]
```

@abstractmethod

```
def word_break(self, s: str, wordDict: list[str]) -> bool:
    """
```

给定一个非空字符串 `s` 和一个包含非空单词的列表 `wordDict`，判定 `s` 是否可以被空格拆分为一个或多个在字典中出现的单词。

说明:

拆分时可以重复使用字典中的单词。你可以假设字典中没有重复的单词。

示例 2:

输入: `s = "applepenapple"`, `wordDict = ["apple", "pen"]`

输出: `true`

"""

`dp = [False]*(len(s) + 1)`

`dp[i]` : 字符串长度为*i*的话, `dp[i]`为`true`, 表示可以拆分为一个或多个在字典中出现的单词。

`dp[0] = True`

遍历背包

`for j in range(1, len(s) + 1):`

 # 遍历单词

`for word in wordDict:`

`if j >= len(word):`

`dp[j] = dp[j] or (dp[j - len(word)] and word == s[j -`

`len(word):j])`

`return dp[len(s)]`

@abstractmethod

`def find_max_form(self, strs: list[str], m: int, n: int) -> int:`

"""

给你一个二进制字符串数组 `strs` 和两个整数 `m` 和 `n` 。

请你找出并返回 `strs` 的最大子集的大小, 该子集中 最多 有 `m` 个 0 和 `n` 个 1 。

如果 `x` 的所有元素也是 `y` 的元素, 集合 `x` 是集合 `y` 的 子集 。

示例 1:

输入: `strs = ["10", "0001", "111001", "1", "0"]`, `m = 5`, `n = 3`

输出: 4

"""

`dp = [[0] * (n + 1) for _ in range(m + 1)]` # 创建二维动态规划数组, 初始化为0

#`dp[i][j]`: 最多有*i*个0和*j*个1的`strs`的最大子集的大小为`dp[i][j]`。

遍历物品

`for s in strs:`

`ones = s.count('1')` # 统计字符串中1的个数

`zeros = s.count('0')` # 统计字符串中0的个数

 # 遍历背包容量且从后向前遍历

`for i in range(m, zeros - 1, -1):`

`for j in range(n, ones - 1, -1):`

`dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)` # 状态

转移方程

`return dp[m][n]`

@abstractmethod

`def num_squares(self, n: int) -> int:`

"""

给定正整数 `n`, 找到若干个完全平方数 (比如 `1, 4, 9, 16, ...`) 使得它们的和等于 `n`。你需要让组成和的完全平方数的个数最少。

给你一个整数 `n` , 返回和为 `n` 的完全平方数的 最少数量 。

示例 1:

输入: `n = 12`

输出: 3

"""

`dp = [float('inf')] * (n + 1)`

`dp[0] = 0`

`for i in range(1, n + 1):` # 遍历背包

```

        for j in range(1, int(i ** 0.5) + 1): # 遍历物品
            # 更新凑成数字 i 所需的最少完全平方数数量
            dp[i] = min(dp[i], dp[i - j * j] + 1)

    return dp[n]

@abstractmethod
def coin_change(self, coins: list[int], amount: int) -> int:
    """
    给你一个整数数组 coins ，表示不同面额的硬币；以及一个整数 amount ，表示总金额。
    计算并返回可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1 。

    输入: coins = [1, 2, 5], amount = 11
    输出: 3
    """
    dp = [float('inf')] * (amount + 1)
    #1. 确定dp数组以及下标的含义: dp[j]: 凑足总额为j所需钱币的最少个数为dp[j]
    dp[0] = 0
    # 3.数组如何初始化
    #如果求组合数就是外层for循环遍历物品，内层for遍历背包。
    #如果求排列数就是外层for遍历背包，内层for循环遍历物品。
    #4.确定遍历顺序: 本题求钱币最小个数，那么钱币有顺序和没有顺序都可以，都不影响钱币的最小
    个数。

```

```

    for i in range(len(coins)):
        for j in range(coins[i], amount + 1):
            dp[j] = min(dp[j], dp[j - coins[i]] + 1)
            #确定递推公式

    if dp[amount] == float('inf'):
        return -1
    return dp[amount]

```

```

@abstractmethod
def rob(self, nums: list[int]) -> int:
    """
    你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在
    同一晚上被小偷闯入，系统会自动报警。给定一个代表每个房屋存放金额的非负整数数组，计算你不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。
    """

```

```

    示例 1:
    输入: [1,2,3,1]
    输出: 4
    """
    dp1 = [0] * len(nums)
    dp0 = [0] * len(nums)
    dp1[0] = nums[0]
    for i in range(1, len(nums)):
        dp1[i] = dp0[i - 1] + nums[i]
        dp0[i] = max(dp1[i - 1], dp0[i - 1])
    return max(dp1[-1], dp0[-1])

```

```

@abstractmethod
def rob2(self, nums: list[int]) -> int:
    """

```


同时，相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你 在不触动警报装置的情况下 ，能够偷窃到的最高金额。

```
输入: nums = [2,3,2]
输出: 3
"""

if not nums: # 如果没有房屋，返回0
    return 0

if len(nums) == 1: # 如果只有一个房屋，返回该房屋的金额
    return nums[0]
dp1 = [0] * (len(nums) - 1)
dp0 = [0] * (len(nums) - 1)
dp1[0] = nums[0]
for i in range(1, len(nums) - 1):
    dp1[i] = dp0[i - 1] + nums[i]
    dp0[i] = max(dp1[i - 1], dp0[i - 1])
curr_max = max(dp1[-1], dp0[-1])

dp1 = [0] * len(nums)
dp0 = [0] * len(nums)
for i in range(1, len(nums)):
    dp1[i] = dp0[i - 1] + nums[i]
    dp0[i] = max(dp1[i - 1], dp0[i - 1])

new_max = max(dp1[-1], dp0[-1])
return max(curr_max, new_max)
```

```
@abstractmethod
def max_profit1(prices: list[int]) -> int:
    """
```

给定一个数组 `prices` ，它的第*i*个元素`prices[i]`表示一支给定股票第*i*天的价格。你只能选择某一天买入这只股票，

并选择在未来的某一个不同的日子卖出该股票。设计一个算法来计算你能获取的最大利润。返回你可以从这笔交易中获取的最大利润。

如果你不能获取任何利润，返回0。

示例 1:

输入: [7,1,5,3,6,4]

输出: 5

"""

"""期间只能购买一次"""

min_val = prices[0]

max_profits = 0

```
for value in prices[1:]:
    if value < min_val:
        min_val = value
    else:
        max_profits = max(max_profits, value - min_val)
return max_profits
```

```
@abstractmethod
```

```
def max_profit2(self, prices: list[int]) -> int:
    """
```

给你一个整数数组 `prices`，其中 `prices[i]` 表示某支股票第 `i` 天的价格。在每一天，你可以决定是否购买和/或出售股票。你在任何时候最多只能持有一股股票。

你也可以先购买，然后在同一天出售。返回你能获得的最大利润。

示例 1:

输入: `prices = [7,1,5,3,6,4]`

输出: 7

```
"""
```

"""期间可以多次购买，但每次只能持有一支股票"""

```
dp_1 = -prices[0] # 持有股票时的最大收益
```

```
dp_0 = 0 # 不持有股票时的最大收益
```

```
for each in prices[1:]:
```

```
    dp_1 = max(dp_1, dp_0 - each)
```

```
    dp_0 = max(dp_0, dp_1 + each)
```

```
return dp_0
```

```
@abstractmethod
```

```
def max_profit3(self, prices: list[int]) -> int:
    """
```

设计一个算法来计算你能获取的最大利润。你最多可以完成 两笔 交易。

```
"""
```

```
if len(prices) == 0:
```

```
    return 0
```

```
dp = [0] * 5
```

```
dp[1] = -prices[0]
```

```
dp[3] = -prices[0]
```

```
for i in range(1, len(prices)):
```

```
    dp[1] = max(dp[1], dp[0] - prices[i])
```

```
    dp[2] = max(dp[2], dp[1] + prices[i])
```

```
    dp[3] = max(dp[3], dp[2] - prices[i])
```

```
    dp[4] = max(dp[4], dp[3] + prices[i])
```

```
return dp[4]
```

```
@abstractmethod
```

```
def max_profit4(self, k: int, prices: list[int]) -> int:
```

""" 设计一个算法来计算你能获取的最大利润。你最多可以完成 `k` 笔交易。 """

```
if len(prices) == 0:
```

```
    return 0
```

```
dp = [[0] * (2*k+1) for _ in range(len(prices))]
```

```
for j in range(1, 2*k, 2):
```

```
    dp[0][j] = -prices[0]
```

```
for i in range(1, len(prices)):
```

```
    for j in range(0, 2*k-1, 2):
```

```
        dp[i][j+1] = max(dp[i-1][j+1], dp[i-1][j] - prices[i])
```

```
        dp[i][j+2] = max(dp[i-1][j+2], dp[i-1][j+1] + prices[i])
```

```
return dp[-1][2*k]
```

```
def max_profit5(self, prices: list[int]) -> int:
    """
```

卖出股票后，你无法在第二天买入股票（即冷冻期为 1 天）。注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入: `prices = [1,2,3,0,2]`

输出: 3

```

"""
dp_0 = -prices[0] # 持有股票时当前的最大收益
dp_1 = 0 # 不持有股票，且处于可交易状态时当前的最大收益
dp_2 = 0 # 不持有股票，且处于冻结状态时当前的最大收益

for each in prices[1:]:
    dp_0 = max(dp_0, dp_1 - each)
    dp_1 = max(dp_1, dp_2)
    dp_2 = dp_0 + each
return max(dp_1, dp_2)

```

@abstractmethod

```

def max_profit6(self, prices: list[int], fee: int) -> int:
    """这里的一笔交易指买入持有并卖出股票的全过程，每笔交易你只需要为支付一次手续费。"""
    dp_1 = -prices[0] # 持有股票时的最大收益
    dp_0 = 0 # 不持有股票时的最大收益
    for each in prices[1:]:
        dp_1 = max(dp_1, dp_0 - each)
        dp_0 = max(dp_0, dp_1 + each - fee)
    return dp_0

```

@abstractmethod

```

def length_of_lis(self, nums: list[int]) -> int:
    if len(nums) <= 1:
        return len(nums)
    dp = [1] * len(nums)
    #dp[i]表示i之前包括i的以nums[i]结尾的最长递增子序列的长度
    result = 1
    for i in range(1, len(nums)):
        for j in range(0, i):
            if nums[i] > nums[j]:
                dp[i] = max(dp[i], dp[j] + 1)
        result = max(result, dp[i]) #取长的子序列
    return result

```

@abstractmethod

```

def find_length_of_lcis(self, nums: list[int]) -> int:
    """
    给定一个未经排序的整数数组，找到最长且 连续递增的子序列，并返回该序列的长度。
    连续递增的子序列 可以由两个下标 l 和 r (l < r) 确定，如果对于每个 l <= i < r，都有
    nums[i] < nums[i + 1] ，
    那么子序列 [nums[l], nums[l + 1], ..., nums[r - 1], nums[r]] 就是连续递增子序
    列。
    """

```

示例 1:

输入: nums = [1,3,5,4,7]

输出: 3

"""

```

dp = [0] * len(nums) # 以下标i为结尾的连续递增的子序列长度为dp[i]。
dp[0] = 1
result = 1
for i in range(1, len(nums)):
    if nums[i] > nums[i - 1]:
        dp[i] = dp[i - 1] + 1
    else:
        dp[i] = 1

```

```
        result = max(result, dp[i])
    return result
```

@abstractmethod

```
def max_sub_array(self, nums: list[int]) -> int:
    """
```

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

输入: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

输出: 6

```
    """
```

```
    dp = [0] * len(nums)
    dp[0] = nums[0]
    result = nums[0]
    for i in range(1, len(nums)):
        if dp[i - 1] < 0:
            dp[i] = nums[i]
        else:
            dp[i] = dp[i - 1] + nums[i]
        result = max(result, dp[i])
    return result
```

@abstractmethod

```
def find_length(self, nums1: list[int], nums2: list[int]) -> int:
    """
```

给两个整数数组 `nums1` 和 `nums2`，返回 两个数组中 公共的、长度最长的子数组的长度。

输入: `nums1 = [1,2,3,2,1]`, `nums2 = [3,2,1,4,7]`

输出: 3

```
    """
```

```
    dp = [[0] * (len(nums2) + 1) for _ in range(len(nums1) + 1)]
    # dp[i][j] : 以下标 i - 1 为结尾的 A，和以下标 j - 1 为结尾的 B，最长重复子数组长度为 dp[i][j]。
    result = 0
```

遍历数组 `nums1`

```
    for i in range(1, len(nums1) + 1):
```

遍历数组 `nums2`

```
        for j in range(1, len(nums2) + 1):
```

如果 `nums1[i-1]` 和 `nums2[j-1]` 相等

```
            if nums1[i - 1] == nums2[j - 1]:
```

在当前位置上的最长公共子数组长度为前一个位置上的长度加一

```
                dp[i][j] = dp[i - 1][j - 1] + 1
```

更新最长公共子数组的长度

```
                if dp[i][j] > result:
```

```
                    result = dp[i][j]
```

返回最长公共子数组的长度

```
    return result
```

@abstractmethod

```
def longest_common_sub_sequence(self, text1: str, text2: str) -> int:
    """
```

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 公共子序列 的长度。如果不存在公共子序列，返回 `0`。

一个字符串的 子序列 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

输入: `text1 = "abcde", text2 = "ace"`

输出: `3`

"""

`dp = [[0] * (len(text2) + 1) for _ in range(len(text1) + 1)]`

`#dp[i][j]: 长度为[0, i - 1]的字符串text1与长度为[0, j - 1]的字符串text2的最长公共子序列为dp[i][j]`

`for i in range(1, len(text1) + 1):`

`for j in range(1, len(text2) + 1):`

`if text1[i-1] == text2[j-1]:`

`# 如果 text1[i-1] 和 text2[j-1] 相等，则当前位置的最长公共子序列长度为左上角位置的值加一`

`dp[i][j] = dp[i - 1][j - 1] + 1`

`else:`

`# 如果 text1[i-1] 和 text2[j-1] 不相等，则当前位置的最长公共子序列长度为上方或左方的较大值`

`dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])`

`return dp[len(text1)][len(text2)]`

`@abstractmethod`

`def max_uncrossed_lines(self, nums1: list[int], nums2: list[int]) -> int:`

`dp = [[0] * (len(nums2)+1) for _ in range(len(nums1)+1)]`

`for i in range(1, len(nums1)+1):`

`for j in range(1, len(nums2)+1):`

`if nums1[i-1] == nums2[j-1]:`

`dp[i][j] = dp[i-1][j-1] + 1`

`else:`

`dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

`return dp[-1][-1]`

`@abstractmethod`

`def is_sub_sequence(self, s: str, t: str) -> bool:`

"""

给定字符串 `s` 和 `t`，判断 `s` 是否为 `t` 的子序列。

输入: `s = "abc", t = "ahbgdc"`

输出: `true`

"""

`dp = [[0] * (len(t)+1) for _ in range(len(s)+1)]`

`for i in range(1, len(s)+1):`

`for j in range(1, len(t)+1):`

`if s[i-1] == t[j-1]:`

`dp[i][j] = dp[i-1][j-1] + 1`

`else:`

`dp[i][j] = dp[i][j-1]`

`if dp[-1][-1] == len(s):`

`return True`

`return False`

`@abstractmethod`

`def count_sub_string(self, s: str) -> int:`

`dp = [[False] * len(s) for _ in range(len(s))]`

dp[i][j]: 表示区间范围[i,j] (注意是左闭右闭)的子串是否是回文子串, 如果是dp[i][j]为true, 否则为false。

```
result = 0
```

所以一定要从下到上, 从左到右遍历, 这样保证dp[i + 1][j - 1]都是经过计算的。

```
for i in range(len(s)-1, -1, -1): #注意遍历顺序
```

```
    for j in range(i, len(s)):
```

```
        if s[i] == s[j]:
```

```
            if j - i <= 1: #情况一 和 情况二
```

```
                result += 1
```

```
                dp[i][j] = True
```

```
            elif dp[i+1][j-1]: #情况三
```

```
                result += 1
```

```
                dp[i][j] = True
```

```
    return result
```

```
@abstractmethod
```

```
def num_distinct(self, s: str, t: str) -> int:
```

```
    """
```

给定一个字符串 s 和一个字符串 t , 计算在 s 的子序列中 t 出现的个数。

输入: s = "rabbbit", t = "rabbit"

输出: 3

```
    """
```

```
    dp = [[0] * (len(t)+1) for _ in range(len(s)+1)]
```

dp[i][j]: 以i-1为结尾的s子序列中出现以j-1为结尾的t的个数为dp[i][j]。简化初始化过程

```
    for i in range(len(s)):
```

```
        dp[i][0] = 1
```

```
    for j in range(1, len(t)):
```

```
        dp[0][j] = 0
```

```
    for i in range(1, len(s)+1):
```

```
        for j in range(1, len(t)+1):
```

```
            if s[i-1] == t[j-1]:
```

```
                dp[i][j] = dp[i-1][j-1] + dp[i-1][j]
```

```
            else:
```

```
                dp[i][j] = dp[i-1][j]
```

```
    return dp[-1][-1]
```

```
@abstractmethod
```

```
def min_distance(self, word1, word2: str) -> int:
```

```
    """
```

给定两个单词 word1 和 word2 , 返回使得 word1 和 word2 相同所需的最小步数。

每步 可以删除任意一个字符串中的一个字符。

输入: word1 = "sea", word2 = "eat"

输出: 2

```
    """
```

```
    dp = [[0] * (len(word2)+1) for _ in range(len(word1)+1)]
```

dp[i][j]: 以i-1为结尾的字符串word1, 和以j-1位结尾的字符串word2, 想要达到相等, 所需要删除元素的最少次数。

```
    for i in range(len(word1)+1):
```

```
        dp[i][0] = i
```

```
    for j in range(len(word2)+1):
```

```
        dp[0][j] = j
```

```
    for i in range(1, len(word1)+1):
```

```
        for j in range(1, len(word2)+1):
```

```
            if word1[i-1] == word2[j-1]:
```

```

        dp[i][j] = dp[i-1][j-1]
    else:
        dp[i][j] = min(dp[i-1][j-1] + 2, dp[i-1][j] + 1, dp[i][j-1] + 1)

    return dp[-1][-1]

```

```

@abstractmethod
def min_distance2(self, word1: str, word2: str) -> int:
    """
    给你两个单词 word1 和 word2， 请返回将 word1 转换成 word2 所使用的最少操作数 。
    你可以对一个单词进行如下三种操作：插入一个字符、删除一个字符、替换一个字符

    输入: word1 = "horse", word2 = "ros"
    输出: 3
    """

    # dp[i][j] 表示以下标i-1为结尾的字符串word1，和以下标j-1为结尾的字符串word2，最近编辑距离为dp[i][j]。
    dp = [[0] * (len(word2)+1) for _ in range(len(word1)+1)]
    for i in range(len(word1)+1):
        dp[i][0] = i
    for j in range(len(word2)+1):
        dp[0][j] = j
    for i in range(1, len(word1)+1):
        for j in range(1, len(word2)+1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                # word2添加一个元素，相当于word1删除一个元素
                # 操作三：替换元，那么只需要一次替换的操作，就可以让 word1[i - 1] 和 word2[j - 1] 相同。
                # 所以 dp[i][j] = dp[i - 1][j - 1] + 1;
                dp[i][j] = min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1]) + 1
    return dp[-1][-1]

```

```

@abstractmethod
def longest_palindrome_subseq(self, s: str) -> int:
    """
    给你一个字符串 s，找出其中最长的回文子序列，并返回该序列的长度。
    子序列定义为：不改变剩余字符顺序的情况下，删除某些字符或者不删除任何字符形成的一个序列。

    输入: s = "bbbab"
    输出: 4
    """

    # dp[i][j]: 字符串s在[i, j]范围内最长的回文子序列的长度为dp[i][j]。
    dp = [[0] * len(s) for _ in range(len(s))]
    for i in range(len(s)):
        dp[i][i] = 1
    for i in range(len(s)-1, -1, -1):
        for j in range(i+1, len(s)):
            if s[i] == s[j]:
                dp[i][j] = dp[i+1][j-1] + 2
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
    return dp[0][-1]

```

```

@abstractmethod

```

```
def longest_palindrome(self, s: str) -> str:
    """
    给你一个字符串 s，找到 s 中最长的回文子串

    输入: s = "babad"
    输出: "bab"
    """
    if len(s) < 2:
        return s
    dp = [[False] * len(s) for _ in range(len(s))]
    result = 1
    result_str = s[0]
    for i in range(len(s) - 1, -1, -1):
        for j in range(i, len(s)):
            if s[i] == s[j]:
                if j - i <= 1:
                    dp[i][j] = True
                elif dp[i + 1][j - 1]:
                    dp[i][j] = True
                if dp[i][j] and j - i + 1 > result:
                    result_str = s[i: j + 1]
                    result = j - i + 1
    return result_str
```

```
from abc import abstractmethod
```

```
class BackTracking:
```

```
    @abstractmethod
```

```
    def combine(self, n: int, k: int) -> list[list[int]]:
```

```
        """
```

```
        给定两个整数 n 和 k，返回 1 ... n 中所有可能的 k 个数的组合。
```

```
        示例：输入：n = 4, k = 2 输出：[ [2,4], [3,4], [2,3], [1,2], [1,3], [1,4],
```

```
        ]
```

```
        """
```

```
    def backtracking(n, k, startIndex, path, result):
```

```
        if len(path) == k:
```

```
            result.append(path[:])
```

```
            return
```

```
        for i in range(startIndex, n + 1): # 需要优化的地方
```

```
            # for i in range(startIndex, n - (k - len(path)) + 2): # 优化的地方，
```

```
            保证有足够的元素满足path的长度
```

```
            path.append(i) # 处理节点
```

```
            self.backtracking(n, k, i + 1, path, result)
```

```
            path.pop() # 回溯，撤销处理的节点
```

```
        result = [] # 存放结果集
```

```
        backtracking(n, k, 1, [], result)
```

```
        return result
```

```
    @abstractmethod
```

```
    def generate_parenthesis(n: int) -> list[str]:
```

```
        """
```

```
        数字 n代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且 有效的 括号组
```

```
        合。
```


示例 1:

输入: `n = 3`

输出: `["((()))","(())","()()", "O(O)", "OOO"]`

示例 2:

输入: `n = 1`

输出: `["O"]`

"""

`ans = []`

```
def backtrack(S, left, right):
    if len(S) == 2 * n:
        ans.append(''.join(S))
        return
    if left < n:
        S.append('(')
        backtrack(S, left + 1, right)
        S.pop()
    if right < left:
        S.append(')')
        backtrack(S, left, right + 1)
        S.pop()
```

`backtrack([], 0, 0)`

`return ans`

@abstractmethod

`def letter_combinations(digits: str) -> list[str]:`

"""

给定一个仅包含数字2-9的字符串，返回所有它能表示的字母组合。答案可以按任意顺序返回。给出数字到字母的映射如下（与电话按键相同）。

注意1不对应任何字母。

示例 1:

输入: `digits = "23"`

输出: `["ad","ae","af","bd","be","bf","cd","ce","cf"]`

"""

```
if not digits:
    return list()
```

```
phone_map = {
    "2": "abc",
    "3": "def",
    "4": "ghi",
    "5": "jkl",
    "6": "mno",
    "7": "pqrs",
    "8": "tuv",
    "9": "wxyz",
}
```

```
def backtrack(index: int):
    if index == len(digits):
        combinations.append(''.join(combination))
    else:
        digit = digits[index]
        for letter in phone_map[digit]:
```

```

        combination.append(letter)
        backtrack(index + 1)
        combination.pop()

```

```

combination = list()
combinations = list()
backtrack(0)
return combinations

```

@abstractmethod

```

def combinationSum3(self, k: int, n: int) -> list[list[int]]:
    """

```

找出所有相加之和为 **n** 的 **k** 个数的组合，且满足下列条件：

只使用数字1到9

每个数字 最多使用一次

返回 所有可能的有效组合的列表 。该列表不能包含相同的组合两次，组合可以以任何顺序返回。

"""

```

def backtracking(k, n, startIndex, path, result):

```

```

    if len(path) == k:

```

```

        if sum(path) == n:

```

```

            result.append(path[:])

```

```

        return

```

```

    for i in range(startIndex, 10):

```

```

        path.append(i)

```

```

        backtracking(k, n, i + 1, path, result)

```

```

        path.pop()

```

```

result = []

```

```

backtracking(k, n, 1, [], result)

```

```

return result

```

@abstractmethod

```

def combinationSum(self, candidates: list[int], target: int) ->
list[list[int]]:
    """

```

给你一个 无重复元素 的整数数组 **candidates** 和一个目标整数 **target** ，找出 **candidates** 中可以使数字和为目标数 **target** 的 所有 不同组合 ，并以列表形式返回。你可以按 任意顺序 返回这些组合。

candidates 中的 同一个 数字可以 无限制重复被选取 。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 **target** 的不同组合数少于 150 个。

示例 1:

输入: **candidates** = [2,3,6,7], **target** = 7

输出: [[2,2,3],[7]]

"""

```

def backtracking(candidates, target, startIndex, path, result):

```

```

    if sum(path) == target:

```

```

        result.append(path[:])

```

```

        return

```

```

    elif sum(path) > target:

```

```

        return

```

```

    else:

```

```

        for i in range(startIndex, len(candidates)):

```

```

            path.append(candidates[i])

```

```

            backtracking(candidates, target, i, path, result)

```

```

        path.pop()

    result = []
    backtracking(candidates, target, 0, [], result)
    return result

@abstractmethod
def combinationSum2(self, candidates: list[int], target: int) ->
list[list[int]]:
    """
    给定一个候选人编号的集合 candidates 和一个目标数 target，找出 candidates 中所有可
    以使数字和为 target 的组合。
    candidates 中的每个数字在每个组合中只能使用 一次 。

    示例 1:
    输入: candidates = [10,1,2,7,6,1,5], target = 8,
    输出:
    [
    [1,1,6],
    [1,2,5],
    [1,7],
    [2,6]
    ]
    """

    def backtracking(candidates, target, startIndex, path, result):
        if sum(path) == target:
            result.append(path[:])
            return
        elif sum(path) > target:
            return
        else:
            for i in range(startIndex, len(candidates)):
                if i > startIndex and candidates[i] == candidates[i - 1]:
                    continue
                path.append(candidates[i])
                backtracking(candidates, target, i+1, path, result)
                path.pop()

    result = []
    candidates.sort()
    backtracking(candidates, target, 0, [], result)
    return result

@abstractmethod
def subsets(self, nums: list[int]) -> list[list[int]]:
    """
    给定一组不含重复元素的整数数组 nums，返回该数组所有可能的子集（幂集）。
    示例：输入: nums = [1,2,3] 输出: [ [3], [1], [2], [1,2,3], [1,3],
    [2,3], [1,2], [] ]
    """

    def backtracking(nums, startIndex, path, result):
        result.append(path[:]) # 收集子集，要放在终止添加的上面，否则会漏掉自己
        # if startIndex >= len(nums): # 终止条件可以不加
        #     return
        for i in range(startIndex, len(nums)):
            path.append(nums[i])
            backtracking(nums, i + 1, path, result)
            path.pop()

```

```

result = []
path = []
backtracking(nums, 0, path, result)
return result

```

@abstractmethod

```

def subsets_with_dup(self, nums: list[int]) -> list[list[int]]:
    """

```

给你一个整数数组 `nums`，其中可能包含重复元素，请你返回该数组所有可能的子集（幂集）。解集不能包含重复的子集。

返回的解集中，子集可以按任意顺序排列。

示例 1:

输入: `nums = [1,2,2]`

输出: `[[], [1], [1,2], [1,2,2], [2], [2,2]]`

"""

```

def backtracking(nums, startIndex, path, result):
    result.append(path[:])
    for i in range(startIndex, len(nums)):
        if i > startIndex and nums[i] == nums[i-1]:
            continue
        path.append(nums[i])
        backtracking(nums, i + 1, path, result)
        path.pop()
    nums.sort()
    result = []
    backtracking(nums, 0, [], result)
    return result

```

@abstractmethod

```

def restore_ip_addresses(self, s: str) -> list[str]:
    """

```

给定一个只包含数字的字符串，复原它并返回所有可能的 IP 地址格式。

有效的 IP 地址 正好由四个整数（每个整数位于 0 到 255 之间组成，且不能含有前导 0），整数之间用 '.' 分隔。

例如: "0.1.2.201" 和 "192.168.1.1" 是 有效的 IP 地址，但是 "0.011.255.245"、"192.168.1.312" 和 "192.168@1.1" 是 无效的 IP 地址。

示例 1:

输入: `s = "25525511135"`

输出: `["255.255.11.135", "255.255.111.35"]`

"""

```

def backtracking(s, start_index, point_num, current, result):
    if point_num == 3: # 逗号数量为3时，分隔结束
        if self.is_valid(s, start_index, len(s) - 1): # 判断第四段子字符串
            current += s[start_index:] # 添加最后一段子字符串
            result.append(current)
        return

```

是否合法

```

    for i in range(start_index, len(s)):
        if self.is_valid(s, start_index, i): # 判断 [start_index, i] 这个

```

区间的子串是否合法

```

        sub = s[start_index:i + 1]
        backtracking(s, i + 1, point_num + 1, current + sub + '.',

```

result)

```

        else:
            break
    result = []
    backtracking(s, 0, 0, "", result)
    return result

```

@abstractmethod

```

def is_valid(self, s, start, end):
    if start > end:
        return False
    if s[start] == '0' and start != end: # 0开头的数字不合法
        return False
    num = 0
    for i in range(start, end + 1):
        if not s[i].isdigit(): # 遇到非数字字符不合法
            return False
        num = num * 10 + int(s[i])
        if num > 255: # 如果大于255了不合法
            return False
    return True

```

@abstractmethod

```

def partition(self, s: str) -> list[list[str]]:
    """

```

给定一个字符串 `s`，将 `s` 分割成一些子串，使每个子串都是回文串。
返回 `s` 所有可能的分割方案。

示例：输入： "aab" 输出： [["aa","b"], ["a","a","b"]]

```

def backtracking(self, s, start_index, path, result ):
    # Base Case
    if start_index == len(s):
        result.append(path[:])
        return

```

单层递归逻辑

```

for i in range(start_index, len(s)):
    # 若反序和正序相同，意味着这是回文串
    if s[start_index: i + 1] == s[start_index: i + 1][::-1]:
        path.append(s[start_index:i+1])
        backtracking(s, i+1, path, result) # 递归纵向遍历：从下一处进行

```

切割，判断其余是否仍为回文串

```

        path.pop() # 回溯
    result = []
    backtracking(s, 0, [], result)
    return result

```

@abstractmethod

```

def find_sub_sequences(self, nums: list[int]) -> list[list[int]]:
    """

```

给你一个整数数组 `nums`，找出并返回所有该数组中不同的递增子序列，递增子序列中 至少有两个元素。你可以按 任意顺序 返回答案。

数组中可能含有重复元素，如出现两个整数相等，也可以视作递增序列的一种特殊情况。

示例 1:

输入： `nums = [4,6,7,7]`

```
输出: [[4,6],[4,6,7],[4,6,7,7],[4,7],[4,7,7],[6,7],[6,7,7],[7,7]]
"""
```

```
def backtracking(nums, startIndex, path, result):
    if len(path) >= 2:
        result.append(path[:])

    uset = set() # 使用集合对本层元素进行去重
    for i in range(startIndex, len(nums)):
        if (path and nums[i] < path[-1]) or nums[i] in uset:
            continue

        uset.add(nums[i]) # 记录这个元素在本层用过了, 本层后面不能再用了
        path.append(nums[i])
        backtracking(nums, i + 1, path, result)
        path.pop()

result = []
backtracking(nums, 0, [], result)
return result
```

@abstractmethod

```
def permute(self, nums: list[int]) -> list[list[int]]:
    """
```

给定一个不含重复数字的数组 `nums` , 返回其 所有可能的全排列 。你可以 按任意顺序 返回答案。

示例 1:

输入: `nums = [1,2,3]`

输出: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

```
"""
```

```
def backtracking(nums, path, result):
    if len(path) == len(nums):
        result.append([nums[i] for i in path])

    for i in range(len(nums)):
        if i in path:
            continue
        path.append(i)
        backtracking(nums, path, result)
        path.pop()

result = []
backtracking(nums, [], result)
return result
```

@abstractmethod

```
def permute_unique(self, nums: list[int]) -> list[list[int]]:
    """
```

给定一个可包含重复数字的序列 `nums` , 按任意顺序 返回所有不重复的全排列。

示例 1:

输入: `nums = [1,1,2]`

输出: `[[1,1,2],[1,2,1],[2,1,1]]`

```
"""
```

```
def backtracking(nums, path, used, result):
    if len(path) == len(nums):
```

```

        result.append(path[:])
        return
    for i in range(len(nums)):
        if (i > 0 and nums[i] == nums[i - 1] and not used[i - 1]) or
used[i]:
            # 保证了对于重复数的集合，一定是从左往右逐个填入的
            continue
        used[i] = True
        path.append(nums[i])
        self.backtracking(nums, path, used, result)
        path.pop()
        used[i] = False
nums.sort() # 排序
result = []
backtracking(nums, [], [False] * len(nums), result)
return result

```

```

from typing import List
from abc import abstractmethod

```

```

class TwoPointer:

```

```

    def sorted_squares(self, nums: List[int]) -> List[int]:
        """

```

977. 有序数组的平方

给你一个按 非递减顺序 排序的整数数组 `nums`，返回 每个数字的平方 组成的新数组，要求也按 非递减顺序 排序。

输入: `nums = [-4,-1,0,3,10]`

输出: `[0,1,9,16,100]`

```

        """

```

```

        result = []

```

```

        low, high = 0, len(nums) - 1

```

```

        while low <= high:

```

```

            if (nums[low])**2 <= (nums[high])**2:

```

```

                result.append((nums[high])**2)

```

```

                high -= 1

```

```

            else:

```

```

                result.append((nums[low])**2)

```

```

                low += 1

```

```

        return result[::-1]

```

```

    def remove_element(self, nums: List[int], val: int) -> int:
        """

```

```

        """

```

27. 移除元素

给你一个数组 `nums` 和一个值 `val`，你需要 原地 移除所有数值等于 `val` 的元素，并返回移除后数组的新长度。

不要使用额外的数组空间，你必须仅使用 $O(1)$ 额外空间并 原地 修改输入数组。

元素的顺序可以改变。你不需要考虑数组中超出新长度后面的元素。

输入: `nums = [3,2,2,3], val = 3`

输出: 2, `nums = [2,2]`

```

        """

```

```

        slow = 0

```

```

for index in range(len(nums)):
    if nums[index] != val: # 所有等于val的数都放在数组的尾部。
        nums[index], nums[slow] = nums[slow], nums[index]
        slow += 1
return slow

```

@abstractmethod

```

def remove_duplicates(nums: list[int]) -> int:
    """

```

给定一个排序数组，你需要在 原地 删除重复出现的元素，使得每个元素只出现一次，返回移除后数组的新长度。

不要使用额外的数组空间，你必须在 原地 修改输入数组 并在使用 $O(1)$ 额外空间的条件下完成。

示例 1:

给定数组 `nums = [1,1,2]`,

函数应该返回新的长度 2, 并且原数组 `nums` 的前两个元素被修改为 1, 2。

"""

```

if not nums:
    return 0
slow = 0
for fast in range(1, len(nums)):
    if nums[fast] != nums[slow]:
        slow += 1
        nums[slow] = nums[fast]
return slow

```

@abstractmethod

```

def reverse_string(self, s: list[str]) -> None:
    """

```

编写一个函数，其作用是将输入的字符串反转过来。输入字符串以字符数组 `s` 的形式给出。不要给另外的数组分配额外的空间，你必须原地修改输入数组、

使用 $O(1)$ 的额外空间解决这一问题。

示例 1:

输入: `s = ["h","e","l","l","o"]`

输出: `["o","l","l","e","h"]`

"""

```

left, right = 0, len(s) - 1

```

```

while left < right:
    s[right], s[left] = s[left], s[right]
    left += 1
    right -= 1

```

@abstractmethod

```

def three_sum(self, nums: List[int]) -> List[List[int]]:
    """

```

15. 三数之和

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足

$nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。注意：答案中不可以包含重复的三元组。

示例 1:

输入: `nums = [-1,0,1,2,-1,-4]`

输出: `[[-1,-1,2],[-1,0,1]]`


```

"""
nums.sort()
result = []
for i in range(len(nums)):
    if nums[i] > 0:
        return result
    if i > 0 and nums[i] == nums[i-1]:
        continue
    left = i + 1
    right = len(nums) - 1
    while left < right:
        if nums[i] + nums[left] + nums[right] == 0:
            result.append([nums[i], nums[left], nums[right]])
            while right > left and nums[right] == nums[right - 1]:
                right -= 1
            while right > left and nums[left] == nums[left + 1]:
                left += 1

            right -= 1
            left += 1
        elif nums[i] + nums[left] + nums[right] < 0:
            left += 1
        else:
            right -= 1
    return result

```

@abstractmethod

```

def four_sum(self, nums: List[int], target: int) -> List[List[int]]:
    """

```

18. 四数之和

给你一个由 n 个整数组成的数组 `nums`，和一个目标值 `target`。请你找出并返回满足下述全部条件且不重复的四元组 `[nums[a], nums[b], nums[c], nums[d]]`

（若两个四元组元素一一对应，则认为两个四元组重复）： $0 \leq a, b, c, d < n$ ， a, b, c 和 d 互不相同， $nums[a] + nums[b] + nums[c] + nums[d] == target$

你可以按 任意顺序 返回答案。

示例 1:

输入: `nums = [1,0,-1,0,-2,2]`, `target = 0`

输出: `[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]`

"""

```

nums.sort()
n = len(nums)
result = []
for i in range(n):
    if nums[i] > target and nums[i] > 0 and target > 0: # 剪枝（可省）
        break
    if i > 0 and nums[i] == nums[i-1]: # 去重
        continue
    for j in range(i+1, n):
        if nums[i] + nums[j] > target and target > 0: # 剪枝（可省）
            break
        if j > i+1 and nums[j] == nums[j-1]: # 去重
            continue
        left, right = j+1, n-1
        while left < right:

```

```

        s = nums[i] + nums[j] + nums[left] + nums[right]
        if s == target:
            result.append([nums[i], nums[j], nums[left],
nums[right]])

            while left < right and nums[left] == nums[left+1]:
                left += 1
            while left < right and nums[right] == nums[right-1]:
                right -= 1
            left += 1
            right -= 1
        elif s < target:
            left += 1
        else:
            right -= 1

    return result

```

@abstractmethod

```

def max_area(height: list[int]) -> int:
    """

```

给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, height[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。返回容器可以储存的最大水量。说明：你不能倾斜容器。

输入: [1,8,6,2,5,4,8,3,7]

输出: 49

"""

```

    left, right = 0, len(height) - 1
    max_val = 0
    while left < right:
        if height[left] <= height[right]:
            max_val = max(max_val, height[left] * (right - left))
            left += 1
        else:
            max_val = max(max_val, height[right] * (right - left))
            right -= 1
    return max_val

```

@abstractmethod

```

def is_palindrome(s: str) -> bool:
    """

```

如果在将所有大写字母转换为小写字母、并移除所有非字母数字字符之后，短语正着读和反着读都一样。则可以认为该短语是一个 回文串 。

字母和数字都属于字母数字字符。给你一个字符串 `s`，如果它是 回文串 ，返回 `true` ；否则，返回 `false` 。

输入: s = "A man, a plan, a canal: Panama"

输出: true

使用头尾双指针，

如果两个指针的元素不相同，则直接返回 `false`，

如果两个指针的元素相同，我们同时更新头尾指针，循环。直到头尾指针相遇。

"""

```

    if not s:
        return True
    left, right = 0, len(s) - 1
    while left < right:

```

```

while left < right and not s[left].isalnum():
    left += 1
while left < right and not s[right].isalnum():
    right -= 1

if s[left].lower() != s[right].lower():
    return False
left += 1
right -= 1
return True

```

```
from abc import abstractmethod
```

```
class MonotonicStack:
```

```
@abstractmethod
```

```
def daily_temperatures(self, temperatures: list[int]) -> list[int]:
    """
```

给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 `i` 天，

下一个更高温度的出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

输入: `temperatures = [73,74,75,71,69,72,76,73]`

输出: `[1,1,4,2,1,1,0,0]`

```
"""
```

```
answer = [0] * len(temperatures)
```

```
stack = []
```

```
for i in range(len(temperatures)):
```

```
    while stack and temperatures[i] > temperatures[stack[-1]]:
```

```
        answer[stack[-1]] = i - stack[-1]
```

```
        stack.pop()
```

```
    stack.append(i)
```

```
@abstractmethod
```

```
def next_greater_element(self, nums1: list[int], nums2: list[int]) ->
list[int]:
    """
```

给你两个 没有重复元素 的数组 `nums1` 和 `nums2`，其中 `nums1` 是 `nums2` 的子集。

请你找出 `nums1` 中每个元素在 `nums2` 中的下一个比其大的值。

`nums1` 中数字 `x` 的下一个更大元素是指 `x` 在 `nums2` 中对应位置的右边的第一个比 `x` 大的元素。如果不存在，对应位置输出 `-1`。

输入: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`。

输出: `[-1,3,-1]`

```
"""
```

```
answer = [-1] * len(nums2)
```

```
stack = []
```

```
for i in range(len(nums2)):
```

```
    while stack and nums2[i] > nums2[stack[-1]]:
```

```
        answer[stack[-1]] = nums2[i]
```

```
        stack.pop()
```

```
    stack.append(i)
```

```
result = []
```

```

for i in range(len(nums1)):
    result.append(answer[nums2.index(nums1[i])])
return result

```

@abstractmethod

```

def next_greater_element2(self, nums: list[int]) -> list[int]:
    """

```

给定一个循环数组（最后一个元素的下一个元素是数组的第一个元素），输出每个元素的下一个更大元素。数字 x 的下一个更大的元素是按数组遍历顺序，

这个数字之后的第一个比它更大的数，这意味着你应该循环地搜索它的下一个更大的数。如果不存在，则输出 -1 。

输入: [1,2,1]

输出: [2,-1,2]

"""

```

dp = [-1] * len(nums)
stack = []
for i in range(len(nums)*2):
    while(len(stack) != 0 and nums[i%len(nums)] > nums[stack[-1]]):
        dp[stack[-1]] = nums[i%len(nums)]
        stack.pop()
    stack.append(i%len(nums))
return dp

```

@abstractmethod

```

def trap(self, height: list[int]) -> int:
    """

```

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

"""

```

stack = [0]
result = 0
for i in range(1, len(height)):
    while stack and height[i] > height[stack[-1]]:
        mid_height = stack.pop()
        if stack:
            # 雨水高度是 min(凹槽左侧高度, 凹槽右侧高度) - 凹槽底部高度
            h = min(height[stack[-1]], height[i]) - height[mid_height]
            # 雨水宽度是 凹槽右侧的下标 - 凹槽左侧的下标 - 1
            w = i - stack[-1] - 1
            # 累计总雨水体积
            result += h * w
        stack.append(i)
return result

```

@abstractmethod

```

def largest_rectangle_area(self, heights: list[int]) -> int:
    """

```

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1 。

求在该柱状图中，能够勾勒出来的矩形的最大面积。

输入: heights = [2,1,5,6,2,3]

输出: 10

"""

```

heights.insert(0, 0)

```

```

heights.append(0)

```

```

stack = [0]
result = 0
for i in range(1, len(heights)):
    while stack and heights[i] < heights[stack[-1]]:
        mid_height = heights[stack[-1]]
        stack.pop()
        if stack:
            # area = width * height
            area = (i - stack[-1] - 1) * mid_height
            result = max(area, result)
    stack.append(i)
return result

```

```

from abc import abstractmethod
from collections import deque
from typing import Optional
from operator import add, sub, mul
import heapq

```

```

class StackAndQueue:

```

```

    @abstractmethod

```

```

    def is_valid(s: str) -> bool:

```

```

        """

```

给定一个只包括 '(', ')', '{', '}', '[', ']' 的字符串 `s`，判断字符串是否有效。

有效字符串需满足：左括号必须用相同类型的右括号闭合。左括号必须以正确的顺序闭合。每个右括号都有一个对应的相同类型的左括号。

```

        输入: s = "()"

```

```

        输出: true

```

```

        """

```

```

        stack = []

```

```

        mapping = {"(": ")", "[": "]", "{": "}"

```

```

        for each in s:

```

```

            if each not in mapping:

```

```

                stack.append(each)

```

```

            else:

```

```

                if not stack or mapping[each] != stack.pop():

```

```

                    return False

```

```

        return len(stack) == 0

```

```

    @abstractmethod

```

```

    def remove_duplicates(self, s: str) -> str:

```

```

        """

```

给出由小写字母组成的字符串 `s`，重复项删除操作会选择两个相邻且相同的字母，并删除它们。

在 `s` 上反复执行重复项删除操作，直到无法继续删除。

```

        输入: "abbaca"

```

```

        输出: "ca"

```

```

        """

```

```

        stack = []

```

```

        for letter in s:

```

```

            if not stack or stack[-1] != letter:

```

```

                stack.append(letter)

```

```

        else:
            stack.pop()
    return ''.join(stack)

```

@abstractmethod

```

def eval_rpn(tokens: list[str]) -> int:
    """

```

150. 逆波兰表达式求值

给你一个字符串数组 `tokens`，表示一个根据 逆波兰表示法 表示的算术表达式。请你计算该表达式的值。返回一个表示表达式值的整数。

输入: `tokens = ["2","1","+","3","*"]`

输出: 9

解释: 该算式转化为常见的中缀算术表达式为: $((2 + 1) * 3) = 9$

```

    """

```

```

    op_map = {'+': add, '-': sub, '*': mul, '/': lambda x, y: int(x / y)}
    stack = []
    for token in tokens:
        if token not in op_map:
            stack.append(int(token))
        else:
            op2 = stack.pop()
            op1 = stack.pop()
            stack.append(op_map[token](op1, op2)) # 第一个出来的在运算符后面
    return stack.pop()

```

@abstractmethod

```

def max_sliding_window(nums: list[int], k: int) -> list[int]:
    """

```

239. 滑动窗口最大值

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 滑动窗口中的最大值 。

示例 1:

输入: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

输出: `[3,3,5,5,6,7]`

```

    """

```

```

    que = MonotonicQueue()
    result = []
    for i in range(k): # 先将前k的元素放进队列
        que.push(nums[i])
    result.append(que.front()) # result 记录前k的元素的最大值
    for i in range(k, len(nums)):
        que.pop(nums[i - k]) # 滑动窗口移除最前面元素
        que.push(nums[i]) # 滑动窗口前加入最后面的元素
        result.append(que.front()) # 记录对应的最大值
    return result

```

@abstractmethod

```

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    """

```

347. 前 K 个高频元素

给你一个整数数组 `nums` 和一个整数 `k`，请你返回其中出现频率前 `k` 高的元素。你可以按 任意顺序 返回答案。

```

输入: nums = [1,1,1,2,2,3], k = 2
输出: [1,2]
"""

# 要统计元素出现频率
map_ = {} # nums[i]:对应出现的次数
for i in range(len(nums)):
    map_[nums[i]] = map_.get(nums[i], 0) + 1

# 对频率排序
# 定义一个小顶堆, 大小为k
pri_que = [] # 小顶堆

# 用固定大小为k的小顶堆, 扫描所有频率的数值
for key, freq in map_.items():
    heapq.heappush(pri_que, (freq, key))
    if len(pri_que) > k: # 如果堆的大小大于了k, 则队列弹出, 保证堆的大小一直为k
        heapq.heappop(pri_que)

# 找出前K个高频元素, 因为小顶堆先弹出的是最小的, 所以倒序来输出到数组
result = [0] * k
for i in range(k - 1, -1, -1):
    result[i] = heapq.heappop(pri_que)[1]
return result

```

```

class QueueUsingStack:

```

```

    """

```

使用栈实现队列的下列操作:

push(x) -- 将一个元素放入队列的尾部。

pop() -- 从队列首部移除元素。

peek() -- 返回队列首部的元素。

empty() -- 返回队列是否为空。

```

    """

```

```

def __init__(self):

```

```

    self.stack = []

```

```

    self.help_stack = []

```

```

def push(self, x: int) -> None:

```

```

    while self.stack:

```

```

        self.help_stack.append(self.stack.pop())

```

```

    self.help_stack.append(x)

```

```

    while self.help_stack:

```

```

        self.stack.append(self.help_stack.pop())

```

```

def pop(self) -> int:

```

```

    return self.stack.pop()

```

```

def peek(self) -> int:

```

```

    return self.stack[-1]

```

```

def empty(self) -> bool:

```

```

    return not bool(self.stack)

```

```
class StackUsingQueue:
```

```
    """
```

225. 用队列实现栈

请你仅使用两个队列实现一个后入先出（LIFO）的栈，并支持普通栈的全部四种操作（push、top、pop 和 empty）。

void push(int x) 将元素 x 压入栈顶。

int pop() 移除并返回栈顶元素。

int top() 返回栈顶元素。

boolean empty() 如果栈是空的，返回 true；否则，返回 false。

```
    """
```

```
def __init__(self) -> None:
```

```
    self.que = deque()
```

```
def push(self, x: int) -> None:
```

```
    self.que.append(x)
```

```
def empty(self) -> bool:
```

```
    return not self.que
```

```
def pop(self) -> Optional[int]:
```

```
    if self.empty():
```

```
        return None
```

```
    for i in range(len(self.que) - 1):
```

```
        self.que.append(self.que.popleft())
```

```
    return self.que.popleft()
```

```
def top(self) -> Optional[int]:
```

```
    if self.empty():
```

```
        return None
```

```
    return self.que[-1]
```

```
class MonotonicQueue:
```

```
def __init__(self) -> None:
```

```
    self.queue = deque()
```

```
def pop(self, value: int) -> None:
```

```
    # 每次弹出的时候，比较当前要弹出的数值是否等于队列出口元素的数值，如果相等则弹出。
```

```
    # 同时pop之前判断队列当前是否为空。
```

```
    if self.queue and value == self.queue[0]:
```

```
        self.queue.popleft()
```

```
def push(self, value: int) -> None:
```

如果push的数值大于入口元素的数值，那么就将队列后端的数值弹出，直到push的数值小于等于队列入口元素的数值为止。

```
    # 这样就保持了队列里的数值是单调从大到小的了。
```

```
    while self.queue and value > self.queue[-1]:
```

```
        self.queue.pop()
```

```
    self.queue.append(value)
```

```
def front(self) -> int:
```

```
    return self.queue[0]
```



```
from abc import abstractmethod
import collections
```

```
class HashTable:
```

```
    @abstractmethod
```

```
    def is_anagram(self, s: str, t: str) -> bool:
```

```
        """
```

242. 有效的字母异位词

给定两个字符串 *s* 和 *t*，编写一个函数来判断 *t* 是否是 *s* 的字母异位词。

注意：若 *s* 和 *t* 中每个字符出现的次数都相同，则称 *s* 和 *t* 互为字母异位词。

```
        """
```

```
        if len(s) != len(t):
```

```
            return False
```

```
        m1 = {}
```

```
        m2 = {}
```

```
        for v1, v2, in zip(s, t):
```

```
            m1[v1] = m1.get(v1, 0) + 1
```

```
            m2[v2] = m2.get(v2, 0) + 1
```

```
        return m1 == m2
```

```
    @abstractmethod
```

```
    def find_anagram(self, s: str, p: str) -> list[int]:
```

```
        """
```

给定两个字符串*s*和*p*，找到*s*中所有*p*的异位词的字串，返回这些子串的起始索引。不考虑答案输出的顺序。异位词指由相同字母重排列形成的字符串

（包括相同的字符串）。

示例1:

输入: *s* = "cbaebabacd", *p* = "abc"

输出: [0,6]

```
        """
```

```
        s_len, p_len = len(s), len(p)
```

```
        if s_len < p_len:
```

```
            return []
```

```
        res = []
```

```
        p_char, s_char = [0] * 26, [0] * 26
```

```
        for index, value in enumerate(p):
```

```
            p_char[ord(value) - ord("a")] += 1
```

```
            s_char[ord(s[index]) - ord("a")] += 1
```

```
        if p_char == s_char:
```

```
            res.append(0)
```

```
        for i in range(s_len - p_len):
```

```
            s_char[ord(s[i]) - ord("a")] -= 1
```

```
            s_char[ord(s[i + p_len]) - ord("a")] += 1
```

```
            if s_char == p_char:
```

```
                res.append(i + 1)
```

```
        return res
```

```
    @abstractmethod
```

```
    def can_construct(self, ransomNote: str, magazine: str) -> bool:
```

```
"""
```

给你两个字符串：ransomNote 和 magazine，判断 ransomNote 能不能由 magazine 里面的字符构成。

如果可以，返回 true；否则返回 false。

magazine 中的每个字符只能在 ransomNote 中使用一次。

输入：ransomNote = "aa", magazine = "ab"

输出：false

```
"""
```

```
ord_list = [0] * 26
```

```
for item in magazine:
    ord_list[ord(item) - ord('a')] += 1
```

```
for item in ransomNote:
    index = ord(item) - ord('a')
    ord_list[index] -= 1
    if ord_list[index] < 0:
        return False
return True
```

```
def is_happy(self, n: int) -> bool:
```

```
"""
```

202. 快乐数

编写一个算法来判断一个数 n 是不是快乐数。

「快乐数」定义为：

对于一个正整数，每一次将该数替换为它每个位置上的数字的平方和。然后重复这个过程直到这个数变为 1，也可能是无限循环但始终变不到 1。

如果这个过程结果为 1，那么这个数就是快乐数。如果 n 是快乐数就返回 true；不是，则返回 false。

```
"""
```

```
record = set()
```

```
while True:
    n = self._get_sum(n)
    if n == 1:
        return True
```

```
# 如果中间结果重复出现，说明陷入死循环了，该数不是快乐数
```

```
if n in record:
    return False
else:
    record.add(n)
```

```
def _get_sum(self, n: int) -> int:
```

```
new_num = 0
while n:
    n, r = divmod(n, 10)
    new_num += r ** 2
return new_num
```

```
@abstractmethod
```

```
def common_chars(self, words: list[str]) -> list[str]:
```

```
"""
```

1002. 查找共用字符

给你一个字符串数组 **words**，请你找出所有在 **words** 的每个字符串中都出现的共用字符（包括重复字符），并以数组形式返回。你可以按任意顺序返回答案。

示例 1:

输入: **words** = ["bella","label","roller"]

输出: ["e","l","l"]

"""

```
if not words: return []
result = []
hash = [0] * 26 # 用来统计所有字符串里字符出现的最小频率
for i, c in enumerate(words[0]): # 用第一个字符串给hash初始化
    hash[ord(c) - ord('a')] += 1
# 统计除第一个字符串外字符的出现频率
for i in range(1, len(words)):
    hashOtherStr = [0] * 26
    for j in range(len(words[i])):
        hashOtherStr[ord(words[i][j]) - ord('a')] += 1
    # 更新hash, 保证hash里统计26个字符在所有字符串里出现的最小次数
    for k in range(26):
        hash[k] = min(hash[k], hashOtherStr[k])
# 将hash统计的字符次数, 转成输出形式
for i in range(26):
    while hash[i] != 0: # 注意这里是while, 多个重复的字符
        result.extend(chr(i + ord('a')))
        hash[i] -= 1
return result
```

@abstractmethod

```
def two_sum(self, nums: list[int], target: int) -> list[int]:
```

"""给定一个整数数组 **nums** 和一个整数目标值 **target**，请你在该数组中找出 和为目标值 **target** 的那两个整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素在答案里不能重复出现。你可以按任意顺序返回答案。

示例 1:

输入: **nums** = [2,7,11,15], **target** = 9

输出: [0,1]"""

value_index = {}

```
for index, value in enumerate(nums):
    if target - value not in value_index:
        value_index[value] = index
    else:
        return [index, value_index[target - value]]
```

@abstractmethod

```
def minimum_recolors(self, blocks: str, k: int) -> int:
```

"""

给你一个长度为 **n** 下标从 0 开始的字符串 **blocks**，**blocks[i]** 要么是 'W' 要么是 'B'，表示第 **i** 块的颜色。字符 'W' 和 'B' 分别表示白色和黑色。

给你一个整数 **k**，表示想要连续黑色块的数目。每一次操作中，你可以选择一个白色块将它涂成黑色块。请你返回至少出现一次连续 **k** 个黑色块的最少操作次数。

示例 1:

输入: **blocks** = "WBBWBBWBW", **k** = 7

输出: 3

```

"""
white = 0
for i in range(k):
    if blocks[i] == "w":
        white += 1
max_val = white

for i in range(0, len(blocks) - k):
    if blocks[i + k] == "w":
        white += 1
    if blocks[i] == "w":
        white -= 1
    max_val = min(max_val, white)
return max_val

```

```

def group_anagrams(self, strs: list[str]) -> list[list[str]]:

```

"""

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。字母异位词是由重新排列源单词的字母得到的一个新单词，所有源单词中的字母通常恰好只用一次。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

"""

```

mapping = {}
for string in strs:
    key, value = self.get_anagrams(string)
    if key in mapping:
        mapping[key].append(value)
    else:
        mapping[key] = [value]
return list(mapping.values())

```

```

def get_anagrams(self, string) -> tuple[tuple[int], str]:
    index = [0 for _ in range(26)]
    for s in string:
        index[ord(s) - 97] += 1
    return tuple(index), string

```

@abstractmethod

```

def first_uniq_char(self, s: str) -> int:

```

"""

给定一个字符串s，找到 它的第一个不重复的字符，并返回它的索引 。如果不存在，则返回 -1。

输入: s = "leetcode"

输出: 0

"""

```

mapping = collections.defaultdict(list)
for index, value in enumerate(s):
    mapping[value].append(index)

result = float('inf')
for key, value in filter(lambda x: len(x[1]) == 1, mapping.items()):
    result = min(result, value[0])
return result if result != float('inf') else -1

```

```

@abstractmethod
def longest_substring(s: str) -> int:
    """
    给定一个字符串 s，请你找出其中不含有重复字符的最长子串的长度。

    输入: s = "abcabcbb"
    输出: 3
    """
    mapping = {}
    start = 0
    max_length = 0
    for index, value in enumerate(s):
        if value in mapping and mapping[value] >= start: # 当一个字符串在新的字
            串中出现过，更新字串的开始位置为，这个字符串之前的下一个位置。
            start = mapping[value] + 1
        max_length = max(max_length, index - start + 1)
        mapping[value] = index
    return max_length

```

```

def quick_sort(array: list[int], left: int, right: int) -> None:
    if left >= right:
        return
    low = left
    high = right
    key = array[low]
    while left < right:
        while left < right and array[right] > key: # 找到第一个小于key的index
            right -= 1
        array[left] = array[right] # 将index移到左边
        while left < right and array[left] <= key: # 找到第一个大于key的index
            left += 1
        array[right] = array[left] # 将index移到右边
    array[left] = key
    quick_sort(array, low, left - 1)
    quick_sort(array, left + 1, high)

if __name__ == "__main__":
    nums = [2, 4, 1, 5, 7, 3, 8, 2]
    quick_sort(nums, 0, len(nums) - 1)
    print(nums)

```

```

from typing import Optional
from abc import abstractmethod
from functools import reduce

class ListNode:
    def __init__(self, val=0, next=None) -> None:
        self.val = val
        self.next = next

class LinkLeetCode:

```

```
@abstractmethod
def remove_element(self, head: Optional[ListNode], val: int) ->
Optional[ListNode]:
    """
    203. 移除链表元素
    给你一个链表的头节点 head 和一个整数 val ，请你删除链表中所有满足 Node.val == val
    的节点，并返回 新的头节点 。
```

示例 1:

输入: head = [1,2,6,3,4,5,6], val = 6

输出: [1,2,3,4,5]

"""

```
if not head:
    return head
pre = ListNode(None, head)
prehead = pre

while head:
    if head.val == val:
        prehead.next = head.next
    else:
        prehead = prehead.next
    head = head.next
return pre.next
```

```
@abstractmethod
def reverse_list(self, head: Optional[ListNode]) -> Optional[ListNode]:
    """
```

给你一个单链表的头节点 head ，请你反转链表，并返回反转后的链表。

示例 1:

输入: head = [1,2,3,4,5]

输出: [5,4,3,2,1]

"""

```
pre = None
while head:
    tmp = head.next
    head.next = pre
    pre = head
    head = tmp
return tmp
```

```
@abstractmethod
def reverse_between(self, head: ListNode, left: int, right: int) -> ListNode:
    """
```

给你一个单链表的头指针 head 和两个整数 left 和 right ，其中 left <= right 。请你反转从位置 left 到位置 right 的链表节点，返回反转后的链表 。

输入: head = [1,2,3,4,5], left = 2, right = 4

输出: [1,4,3,2,5]

"""

```
cur = ListNode(None, head)
pre = cur
count = 0
while count < left - 1:    # head指向需要reverse的左边
    pre = pre.next
```

```

        head = head.next
        count += 1
    mark_left = head

    reverse = None
    while count < right: # head指向right的下一个节点
        tmp = head.next
        head.next = reverse
        reverse = head
        head = tmp

    mark_left.next = head
    pre.next = reverse
    return cur.next

```

```

def reverse_pairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
    """

```

24. 两两交换链表中的节点

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的情况下完成本题（即，只能进行节点交换）。

示例 1:

输入: head = [1,2,3,4]

输出: [2,1,4,3]

"""

```

if not head or not head.next:
    return head

```

```

head1 = head.next
head2 = head.next.next
head1.next = head
head.next = self.reverse_pairs(head2)
return head1

```

```

@abstractmethod

```

```

def remove_nth_from_end(self, head: Optional[ListNode], n: int) ->
Optional[ListNode]:
    """

```

给你一个链表，删除链表的倒数第n个结点，并且返回链表的头结点。

示例 1:

输入: head = [1,2,3,4,5], n = 2

输出: [1,2,3,5]

示例 2:

输入: head = [1], n = 1

输出: []

"""

```

pre = ListNode(None, head)
cur = pre
for i in range(n): # 先走n步
    head = head.next
while head:
    head = head.next
    pre = pre.next
pre.next = pre.next.next
return cur.next

```

```

@abstractmethod
def get_intersection_node(self, head1: ListNode, head2: ListNode) ->
Optional[ListNode]:
    """
    给你两个单链表的头节点 headA 和 headB ，请你找出并返回两个单链表相交的起始节点。如果
    两个链表不存在相交节点，返回 null 。
    """
    head_a, head_b = head1, head2
    while head_a != head_b:
        head_a = head_a.next if head_a else head2
        head_b = head_b.next if head_b else head1
    return head_a

```

给你两个单链表的头节点 **headA** 和 **headB** ，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 **null** 。

```

@abstractmethod
def add_two_numbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
    """
    给你两个非空的链表，表示两个非负的整数。它们每位数字都是按照逆序的方式存储的，并且每个节
    点只能存储一位数字。
    请你将两个数相加，并以相同形式返回一个表示和的链表。你可以假设除了数字 0 之外，这两个数
    都不会以0开头。
    示例 1:
    输入: l1 = [2,4,3], l2 = [5,6,4]
    输出: [7,0,8]
    """
    flag = 0
    pre = ListNode(0, l1) # l1作为输出
    res = pre
    while l1 is not None or l2 is not None: # 如果l1或者l2没有到尾节点
        if l1 is not None:
            flag += l1.val
            l1 = l1.next
        if l2 is not None:
            flag += l2.val
            l2 = l2.next
        flag, x = divmod(flag, 10)
        if pre.next is not None: # 如果l1没到尾节点，直接赋值，否则新建节点，当l1走
            pre.next.val = x
        else:
            pre.next = ListNode(x, None)
        pre = pre.next
    if flag == 1:
        pre.next = ListNode(flag, None)
    return res.next

```

给你两个非空的链表，表示两个非负的整数。它们每位数字都是按照逆序的方式存储的，并且每个节点只能存储一位数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。你可以假设除了数字 0 之外，这两个数都不会以0开头。

示例 1:

输入: l1 = [2,4,3], l2 = [5,6,4]

输出: [7,0,8]

```

@abstractmethod
def partition(self, head: Optional[ListNode], x: int) -> Optional[ListNode]:
    """
    给你一个链表的头节点 head 和一个特定值 x ，请你对链表进行分隔，使得所有 小于 x 的节点
    都出现在 大于或等于 x 的节点之前。
    你应当 保留 两个分区中每个节点的初始相对位置。
    示例 1:
    输入: head = [1,4,3,2,5,2], x = 3

```



```
输出: [1,2,2,4,3,5]
"""
```

```
pre1 = ge = ListNode()
pre2 = le = ListNode()
while head:
    if head.val < x:
        ge.next = head
        ge = ge.next
    else:
        le.next = head
        le = le.next
    head = head.next

le.next = None
ge.next = pre2.next
return pre1.next
```

@abstractmethod

```
def reorder_list(self, head: Optional[ListNode]) -> Optional[ListNode]:
    """
```

给定一个单链表 L 的头节点 $head$ ，单链表 L 表示为:

$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

请将其重新排列后变为:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

示例 1:

输入: $head = [1,2,3,4]$

输出: $[1,4,2,3]$

```
"""
```

```
if not head:
    return
```

```
vec = list()
node = head
while node:
    vec.append(node)
    node = node.next
```

```
i, j = 0, len(vec) - 1
while i < j:
    vec[i].next = vec[j]
    i += 1
    if i == j:
        break
    vec[j].next = vec[i]
    j -= 1
    if i == j:
        break
```

```
vec[i].next = None
return vec[0]
```

@abstractmethod

```
def delete_duplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
```

```
"""
```

给定一个已排序的链表的头 **head**，删除所有重复的元素，使每个元素只出现一次。返回 已排序的链表。

示例 1:

输入: **head** = [1,1,2]

输出: [1,2]

```
"""
```

```
if not head:
    return head
cur = head
while head.next:
    if head.val == head.next.val:
        head.next = head.next.next
    else:
        head = head.next
return cur
```

```
@abstractmethod
```

```
def delete_duplicates_v2(head: Optional[ListNode]) -> Optional[ListNode]:
    """
```

给定一个已排序的链表的头 **head**，删除原始链表中所有重复数字的节点，只留下不同的数字。返回 已排序的链表。

示例 1:

输入: **head** = [1,2,3,3,4,4,5]

输出: [1,2,5]

```
"""
```

```
if not head:
    return head

pre = ListNode(0, head)
res = pre
while pre.next and pre.next.next:
    if pre.next.val == pre.next.next.val:
        val = pre.next.val
        while pre.next and pre.next.val == val:
            pre.next = pre.next.next
    else:
        pre = pre.next
return res.next
```

```
@abstractmethod
```

```
def has_cycle(head: Optional[ListNode]) -> bool:
    """
```

给你一个链表的头节点 **head**，判断链表中是否有环。如果链表中存在某个节点，可以通过连续跟踪 **next** 指针再次到达，则链表中存在环。如果链表中不存在环，则返回 **true**。否则，返回 **false**。

示例 1:

输入: **head** = [3,2,0,-4], **pos** = 1

输出: **true**

解释: 链表中有一个环，其尾部连接到第二个节点。

```
"""
```

```
if not head:
    return False
slower = head
```

```

faster = head
while faster and faster.next:
    slower = slower.next
    faster = faster.next.next
    if slower == faster:
        return True
return False

```

@abstractmethod

```

def detect_cycle(self, head: ListNode) -> ListNode:
    """

```

假设从头结点到环形入口节点的节点数为 x 。 环形入口节点到 **fast**指针与**slow**指针相遇节点 节点数为 y 。 从相遇节点 再到环形入口节点节点数为 z 。

那么相遇时: **slow**指针走过的节点数为: $x + y$, **fast**指针走过的节点数: $x + y + n (y + z)$, n 为**fast**指针在环内走了 n 圈才遇到**slow**指针, $(y+z)$ 为 一圈内节点的个数 A 。

因为**fast**指针是一步走两个节点,**slow**指针一步走一个节点, 所以 **fast**指针走过的节点数 = **slow**指针走过的节点数 * 2:

$$(x + y) * 2 = x + y + n (y + z)$$

$$\text{两边消掉一个 } (x+y) : x + y = n (y + z)$$

因为要找环形的入口, 那么要求的是 x , 因为 x 表示 头结点到 环形入口节点的的距离。

$$\text{所以要求 } x, \text{ 将 } x \text{ 单独放在左面: } x = n (y + z) - y ,$$

再从 $n(y+z)$ 中提出一个 $(y+z)$ 来, 整理公式之后为如下公式: $x = (n - 1) (y + z) + z$
注意这里 n 一定是大于等于1的, 因为 **fast**指针至少要多走一圈才能相遇**slow**指针。

```

    """
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        # If there is a cycle, the slow and fast pointers will eventually
meet
        if slow == fast:
            # Move one of the pointers back to the start of the list
            slow = head
            while slow != fast:
                slow = slow.next
                fast = fast.next
            return slow
        # If there is no cycle, return None
    return None

```

@abstractmethod

```

def merge_two_link_list(self, list1: Optional[ListNode], list2:
Optional[ListNode]) -> Optional[ListNode]:
    """

```

将两个升序链表合并为一个新的 升序 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1:

输入: l1 = [1,2,4], l2 = [1,3,4]

输出: [1,1,2,3,4,4]

"""

```
pre = ListNode()
```

```
res = pre
```

```
while list1 is not None and list2 is not None:
```

```
    tmp = None
```

```
    if list1.val <= list2.val:
```

```
        tmp = list1
```

```
        list1 = list1.next
```

```
    else:
```

```
        tmp = list2
```

```
        list2 = list2.next
```

```
    tmp.next = None # 切断连接
```

```
    res.next = tmp
```

```
    res = res.next
```

```
res.next = list2 if list2 is not None else list1
```

```
return pre.next
```

```
def merge_two_link_list_v2(self, l1: ListNode, l2: ListNode) -> ListNode:
```

```
    if not l1: return l2 # 终止条件, 直到两个链表都空
```

```
    if not l2: return l1
```

```
    if l1.val <= l2.val: # 递归调用
```

```
        l1.next = self.merge_two_link_list_v2(l1.next, l2)
```

```
        return l1
```

```
    else:
```

```
        l2.next = self.merge_two_link_list_v2(l1, l2.next)
```

```
        return l2
```

"""

给你一个链表数组，每个链表都已经按升序排列。请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1:

输入: lists = [[1,4,5],[1,3,4],[2,6]]

输出: [1,1,2,3,4,4,5,6]

示例 2:

输入: lists = []

输出: []

"""

```
def merge_k_link_list(lists: list[Optional[ListNode]]) -> Optional[ListNode]:
```

```
    if not lists:
```

```
        return None
```

```
    return reduce(lambda x, y: LinkLeetCode.merge_two_link_list(x, y), lists)
```

"""

给你两个按 非递减顺序 排列的整数数组 nums1 和 nums2，另有两个整数 m 和 n，分别表示 nums1 和 nums2 中的元素数目。

请你 合并 `nums2` 到 `nums1` 中，使合并后的数组同样按 非递减顺序 排列。

注意：最终，合并后数组不应由函数返回，而是存储在数组 `nums1` 中。为了应对这种情况，`nums1` 的初始长度为 $m + n$ ，其中前 m 个元素表示应合并的元素，后 n 个元素为 0，应忽略。`nums2` 的长度为 n 。

示例 1:

输入: `nums1 = [1,2,3,0,0,0]`, $m = 3$, `nums2 = [2,5,6]`, $n = 3$

输出: `[1,2,2,3,5,6]`

"""

```
def merge(nums1: list[int], m: int, nums2: list[int], n: int) -> None:
```

```
    """
```

```
    Do not return anything, modify nums1 in-place instead.
```

```
    """
```

```
    pos = m + n - 1
```

```
    while m > 0 and n > 0:
```

```
        if nums1[m - 1] < nums2[n - 1]:
```

```
            nums1[pos] = nums2[n - 1]
```

```
            n -= 1
```

```
        else:
```

```
            nums1[pos] = nums1[m - 1]
```

```
            m -= 1
```

```
        pos -= 1
```

```
    while n > 0:
```

```
        nums1[pos] = nums2[n - 1]
```

```
        n -= 1
```

```
        pos -= 1
```


